

Advanced C Programming Lab

Session VII – Apr 30

Objectives

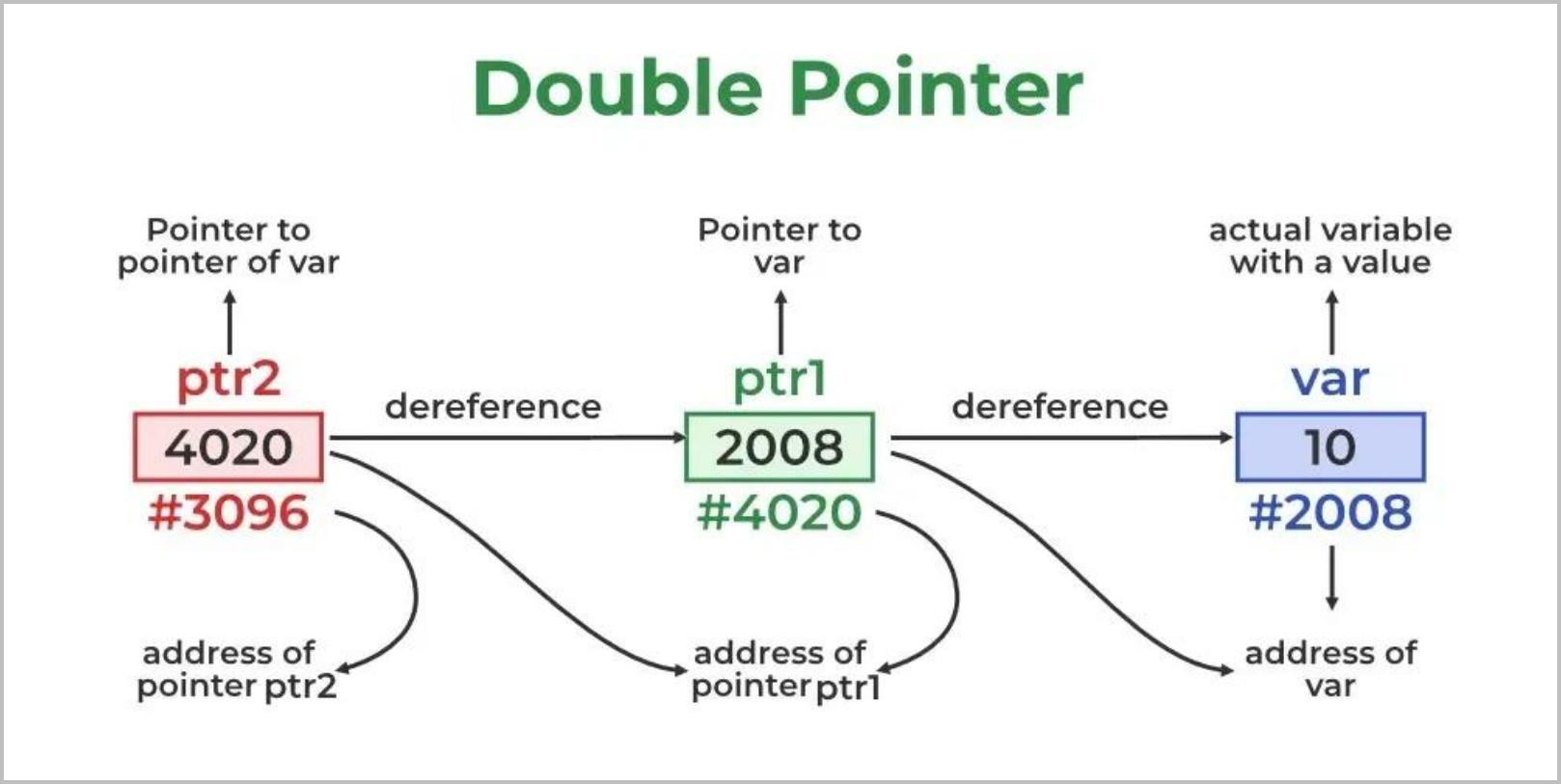
We will be talking about;

- Pointers to Pointers

In C, we can also have a pointer that points to another pointer which is called pointer to pointer (Double Pointer).

Pointer to pointer

Simply put, a normal pointer is like a note that has the address on it. A double pointer is another note that can tell you where the first note is kept.



Pointer to pointer

```c
#include <stdio.h>

int main() {
  int Num = 12;       // normal variable
  int *ptr = &Num;    // pointer to int
  int **pptr = &ptr;   // pointer to pointer

  printf("Num = %d\n", Num);
  printf("*ptr = %d\n", *ptr);
  printf("**pptr = %d\n", **pptr);

  return 0;
}
```

Output:

Num = 12
*ptr = 12
**pptr = 12

Pointer to pointer

```
int Num = 12;
 int *ptr = &Num;
  int **pptr = &ptr;
```

- `Num` holds the value `12`.
- `ptr` holds the address of `Num`.
- `pptr` holds the address of `ptr`.
- `*ptr` gives the value of `Num`.
- `**pptr` also gives the value of `Num`, by going through both pointers.

Pointer to pointer

```c
#include <stdio.h>

int main() {
  int Num = 12;
  int *ptr = &Num;
  int **pptr = &ptr;

  printf("Old Num = %d\n", Num);

  **pptr = 34;    // changes Num

  printf("New Num = %d\n", Num);
  return 0;
}
```

Output:

Old Num = 12
New Num = 34

Pointer to pointer

Passing Array of Strings to Function: Array of strings are generally stored as array of pointer to strings. This can be passed using double pointers to function..

```
#include <stdio.h>
// Function that takes array of strings as argument
void print(char **arr, int n) {
    for (int i = 0; i < n; i++)
        printf("%s\n", *(arr + i));
}
int main() {
    char *arr[10] = {"Hi", "Hello", "Salut", "Bonjour"};
    print(arr, 3);
    return 0;
}
```

Output:

Hi
Hello
Salut

Pointer to pointer

Size of a double pointer is same as size of any other pointer as it also stored address.

```
#include <stdio.h>


int main(){
    // Defining single and double pointers
    int a = 7;
    int *ptr = &a;
    int **d_ptr = &ptr;
    // Size of double pointer
    printf("%d bytes\n", sizeof(d_ptr));
    // Size of a single pointer
    printf("%d bytes", sizeof(ptr));
    return 0;
}
```

Output:

```
8 bytes
8 bytes
```

Thank you for your time and attention!

References:

- https://www.w3schools.com/
- https://www.geeksforgeeks.org/
- https://www.tutorialspoint.com/